

Documento de Diseño del Sistema (SDD)
Plataforma de Agendamiento Estricto para Barbería

Contenido

Documento de Diseño del Sistema (SDD).....	1
Plataforma de Agendamiento Estricto para Barbería	1
1. Arquitectura del Sistema	2
2. Pila Tecnológica (Tech Stack).....	2
3. Modelo de Datos (Esquema de Base de Datos Relacional)	2
4. Diseño del Motor de Notificaciones (El "Worker" de 5 minutos)	3
5. Diseño de Endpoints (API RESTful)	4
6. Flujo de Interfaz de Usuario (UI/UX) en React / Ionic	5
7. Estrategia de Despliegue y CI/CD	5

1. Arquitectura del Sistema

Para la arquitectura de este proyecto, se estructurará como un Sistema Monolítico Modular. Esta aproximación arquitectónica centraliza la lógica de negocio en una única base de código, lo cual es ideal para mantener un control transaccional estricto y evitar la complejidad innecesaria de los microservicios en etapas iniciales. Todo el flujo de datos (agendamiento, validación y notificaciones) residirá en el mismo entorno, asegurando que la base de datos sea la única fuente de verdad sin problemas de sincronización distribuida.

2. Pila Tecnológica (Tech Stack)

- **Frontend Móvil (Cliente y Peluquero):** Ionic Framework con React. Permitirá compilar para iOS y Android desde un solo código fuente, utilizando componentes web adaptables.
- **Backend (API RESTful):** Node.js con Express. Proveerá la lógica de negocio y la comunicación con la base de datos.
- **Base de Datos:** PostgreSQL alojada en Supabase. Garantiza integridad referencial y bloqueos a nivel de fila (Row-Level Locking), crucial para el manejo de reservas concurrentes.
- **Infraestructura / Despliegue:** Despliegue del backend en Fly.io o Koyeb para baja latencia.

3. Modelo de Datos (Esquema de Base de Datos Relacional)

El manejo del tiempo y las zonas horarias es fundamental. Todas las fechas deben guardarse estrictamente en formato UTC en la base de datos y solo transformarse a la zona horaria local (ej. America/Bogota) en el frontend de Ionic.

Tablas Principales:

Usuarios		
id	UUID	Primary Key
nombre	Varchar	
email	Varchar	Unique
telefono	Varchar	
rol	Enum: 'CLIENTE', 'PELUQUERO', 'ADMIN'	

Servicios		
id	UUID	Primary Key
nombre_servicio	Varchar	
duración_minutos	Integer	
precio	Decimal	

Citas		
id	UUID	Primary Key
Código_verificacion	Varchar	Unique, Indexado
Cliente_id	UUID	Foreign Key - usuarios
Peluquero_id	UUID	Foreign Key – usuarios
Inicio_esperado	Timestamp	
Fin esperado	timestamp	
estado	Enum: 'AGENDADA', 'EN_PROGRESO', 'FINALIZADA', 'CANCELADA', 'NO_ASISTIO'	
notificacion_enviada	Boolean	Default: false

4. Diseño del Motor de Notificaciones (El "Worker" de 5 minutos)

Para cumplir con el requisito crítico de notificar exactamente 5 minutos antes del turno, el diseño implementará un proceso en segundo plano (Background Job):

1. **El Cron Job:** Un temporizador en el servidor de Node.js ejecutará una consulta a PostgreSQL cada 1 minuto (ej. usando la librería node-cron).
2. **La Consulta:** Buscará en la tabla Citas donde:
 - estado = 'AGENDADA'
 - notificacion_enviada = false
 - inicio_esperado <= (Hora Actual + 5 minutos) y inicio_esperado > Hora Actual.
3. **Ejecución:** Por cada registro encontrado, el backend enviará el *payload* a Firebase Cloud Messaging apuntando al token del dispositivo del cliente_id.
4. **Actualización:** Inmediatamente después, el servidor marca el campo notificacion_enviada como true para asegurar que el cliente no reciba spam en el siguiente minuto.

5. Diseño de Endpoints (API RESTful)

Aquí se definen los contratos principales entre Ionic-React y Node.js:

- **POST /api/v1/citas**
 - **Propósito:** Agendar una nueva cita.
 - **Lógica Crítica:** Antes de insertar, el sistema ejecuta un SELECT ... FOR UPDATE o verifica transaccionalmente que no existan registros en la tabla Citas para ese peluquero_id cuyo rango de inicio_esperado y fin_esperado se superponga con el rango solicitado.
- **GET /api/v1/citas/disponibilidad**
 - **Parámetros:** fecha, peluquero_id.
 - **Retorno:** Un arreglo con los bloques de tiempo libres calculados a partir de los horarios de atención y las citas ya existentes en estado 'AGENDADA' o 'EN_PROGRESO'.
- **POST /api/v1/citas/verificar**
 - **Propósito:** Endpoint exclusivo para el peluquero.
 - **Cuerpo (Body):** { "codigo_verificacion": "BARB-8A2F" }
 - **Lógica:** Valida si el código existe, si corresponde al día de hoy, y devuelve los datos del cliente para que el peluquero confirme visualmente.
- **PATCH /api/v1/citas/:id/estado**
 - **Propósito:** Transicionar la cita.
 - **Cuerpo (Body):** { "nuevo_estado": "FINALIZADA" }
 - **Lógica:** Actualiza el registro. Si pasa a "FINALIZADA", el sistema libera el turno en la interfaz del peluquero y guarda el registro para el historial de contabilidad.

6. Flujo de Interfaz de Usuario (UI/UX) en React / Ionic

Dado que se utilizará React para construir la interfaz de Ionic, el estado de la aplicación se manejará de forma estructurada. Las vistas principales serán:

- **Aplicación del Cliente:**
 - **Pantalla Home / Dashboard:** Muestra un botón de acción principal ("Agendar Nuevo Corte") y una tarjeta resaltada si el usuario tiene una cita próxima, mostrando un contador de tiempo regresivo.
 - **Pantalla de Calendario:** Un selector de fecha y bloques de horas. Los bloques ocupados deben tener un estilo visual distintivo (ej. color gris y no clickeables).
 - **Pantalla de Confirmación (El Ticket):** Muestra el **ID de Turno** en tipografía grande, la hora exacta y las instrucciones de llegada.
- **Aplicación / Panel del Peluquero:**
 - **Vista de Agenda Diaria:** Una lista vertical cronológica de los turnos del día actual. Cada ítem muestra el estado actual mediante colores (ej. Amarillo = Agendada, Verde = En Progreso, Gris = Finalizada).
 - **Modal de Verificación de ID:** Un campo de texto rápido (o escáner si se usa cámara) para ingresar el código del cliente y habilitar el botón "Iniciar Corte".

7. Estrategia de Despliegue y CI/CD

- **Repositorio:** El código estará alojado en GitHub.
- **Backend (Node.js):** Se conectará el repositorio directamente a la plataforma de despliegue elegida (ej. Koyeb o Fly.io). Cada vez que se integre código a la rama main, se disparará un despliegue automático.
- **Base de Datos (Supabase):** Las migraciones de la base de datos (creación de tablas Usuarios, Citas, etc.) se manejarán mediante scripts SQL versionados para mantener el control sobre los esquemas.
- **Frontend (Ionic):** Durante la fase de pruebas, la versión web (PWA) puede alojarse en Vercel o Netlify para compartir avances rápidos. Para producción, se compilarán los binarios (APK para Android, IPA para iOS) usando el CLI de Ionic.